Ex No: 02

END-TO-END COMMUNICATION AT TRANSPORT LAYER

A) Implement echo client server using TCP/UDP Socket.

AIM: To implement an echo client server program using TCP/UDP Sockets.

ALGORITHM:
- Create a Server Socket
- Bind the socket to an IP Address and port.
- Listen for client connection
- Accept a client connection
- Receive a message from the client
- Display the echoed message

CODE:

```
import socket
server = socket.Socket(Socket.AF_INET, Socket.Sock_STREAM)
server.listen(1)
print("Server waiting for client....")
conn, addr = server.accept()
print("Connected:", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    conn.sendall(data)
conn.(0 sec)
conn.close()
```

RESULT: Thus, the echo client-server program using TCP/UDP socket is implemented successfully.

B) Implement a chat program using socket programming.

AIM: To implement a simple client-server chat program using TCP socket programming.

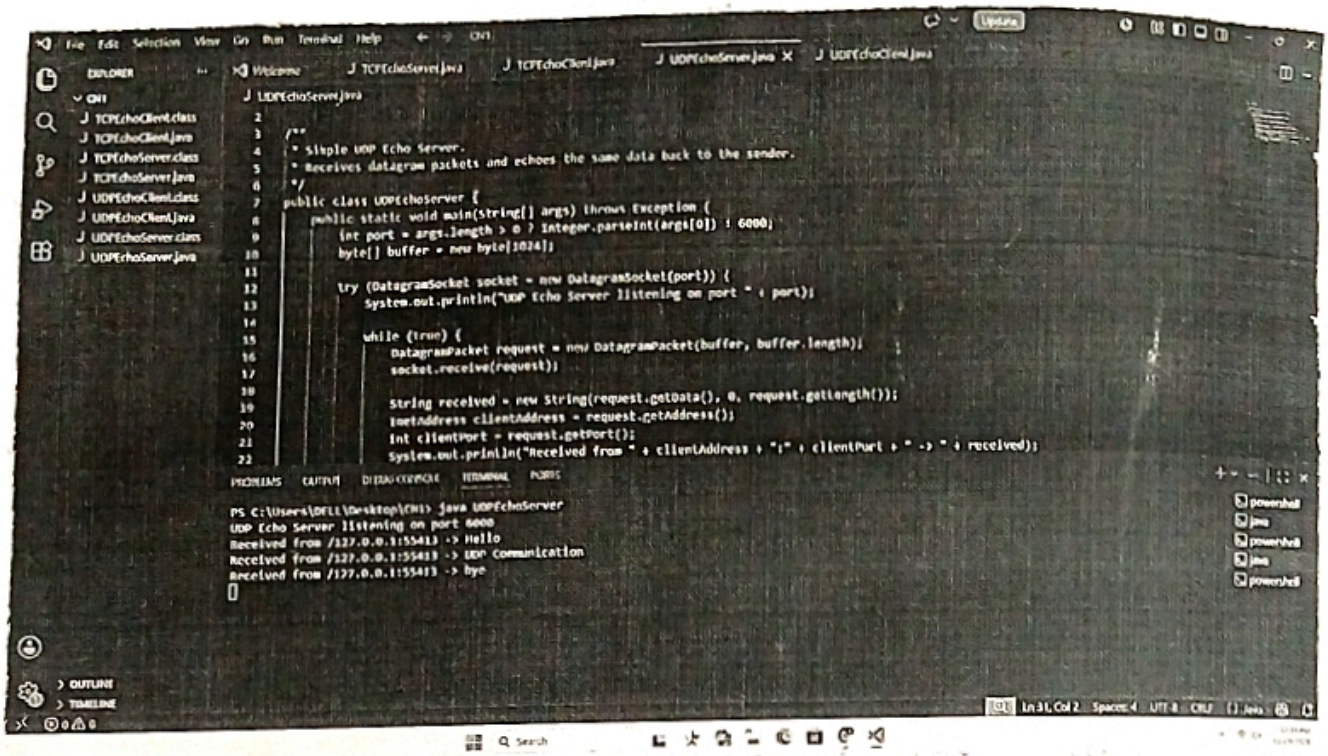
PROCEDURE:

* Run the server program first.
* Run the client program in another terminal.
* Exchange message between the client and Server.
* Type bye to end the chat.

CODE:

```
import socket
server = socket.socket()
server.bind(("127.0.0.1", 6000))
server.listen(1)
print("Waiting for client...")
conn,addr = server.accept()
print("Client connected")
while True:
    msg = conn.recv(1024).socket()
    if msg.lower() == "bye":
        break
    print("Client:",msg)
    reply = input("Server:")
    conn.sendall(reply.encode())
    if reply.lower() == "bye":
        break
conn.close()
server.close()
```

RESULT: A client - server chat program is implemented successfully using TCP socket.



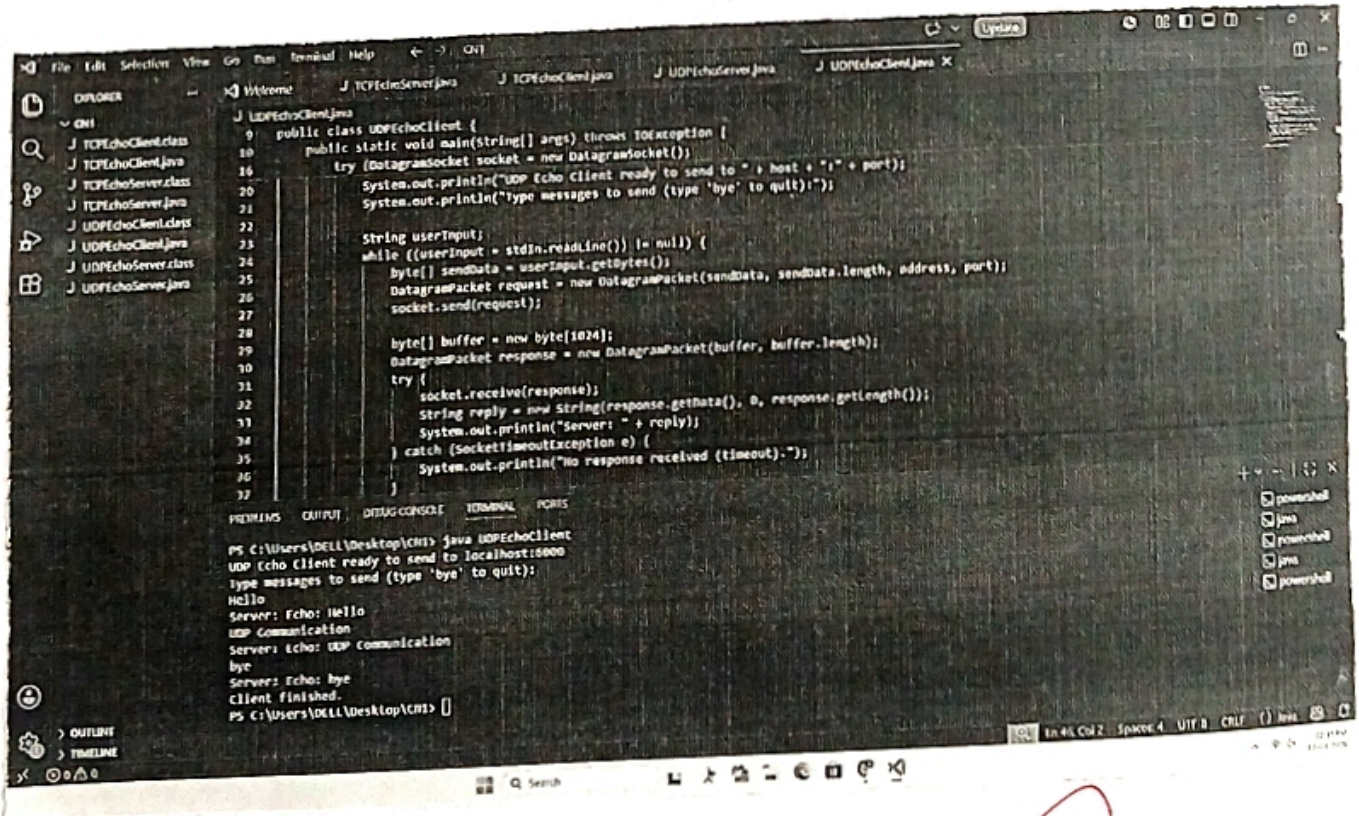
```java
/**
 * Simple UDP Echo Server.
 * Receives datagram packets and echoes the same data back to the sender.
 */
public class UDPEchoServer {
    public static void main(String[] args) throws Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
        byte[] buffer = new byte[1024];

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Echo Server listening on port " + port);

            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                String received = new String(request.getData(), 0, request.getLength());
                InetAddress clientAddress = request.getAddress();
                int clientPort = request.getPort();
                System.out.println("Received from " + clientAddress + ":" + clientPort + " -> " + received);
```

```
PS C:\Users\DELL\Desktop\CN1> java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:55413 -> Hello
Received from /127.0.0.1:55413 -> UDP Communication
Received from /127.0.0.1:55413 -> bye
```

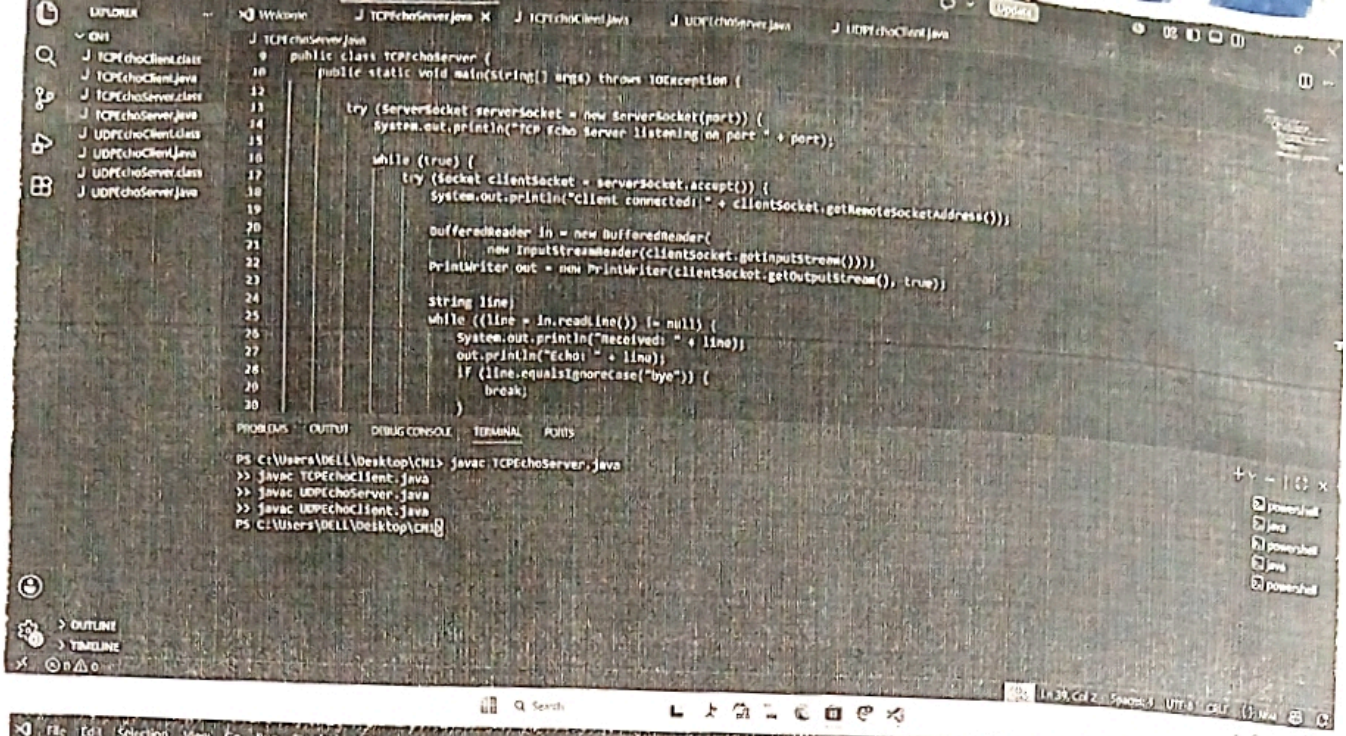


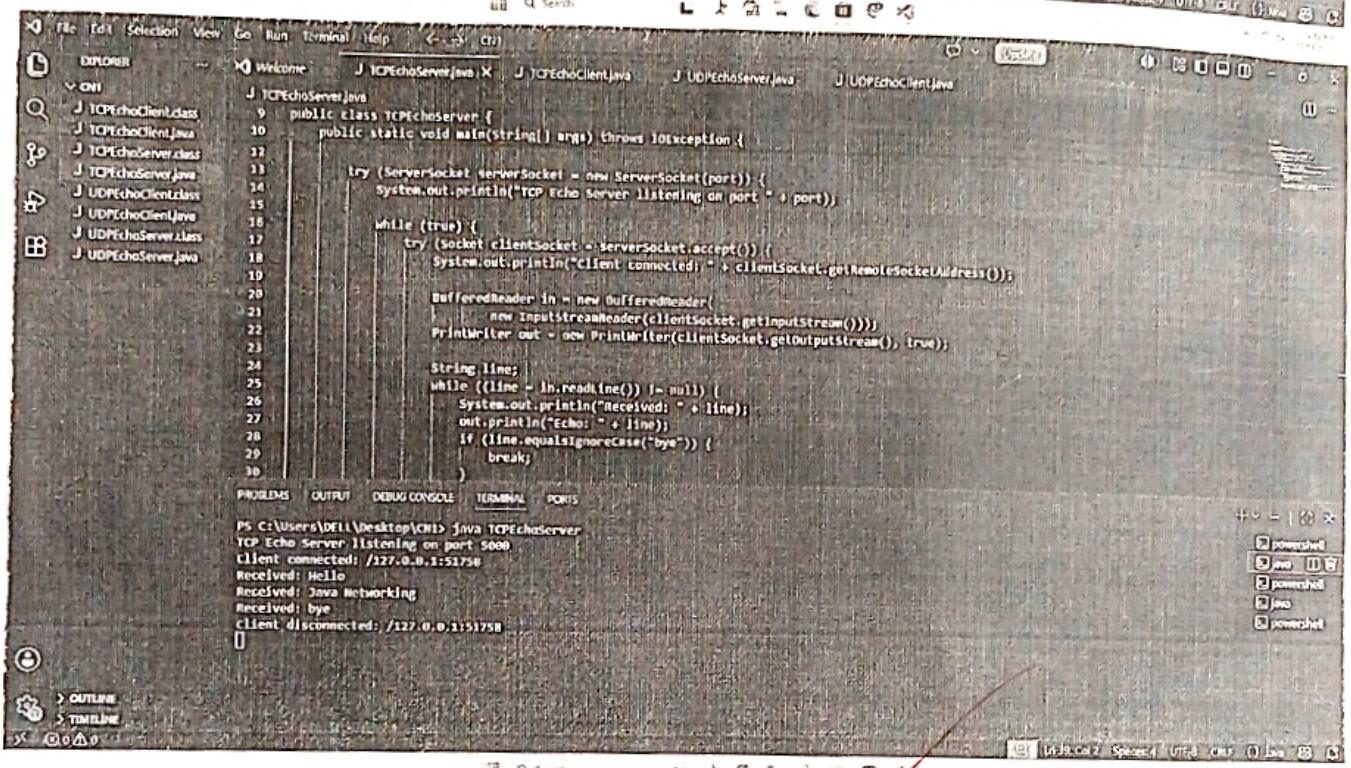
```java
public class UDPEchoClient {
    public static void main(String[] args) throws IOException {
        try (DatagramSocket socket = new DatagramSocket()) {
            System.out.println("UDP Echo Client ready to send to " + host + ":" + port);
            System.out.println("Type messages to send (type 'bye' to quit):");

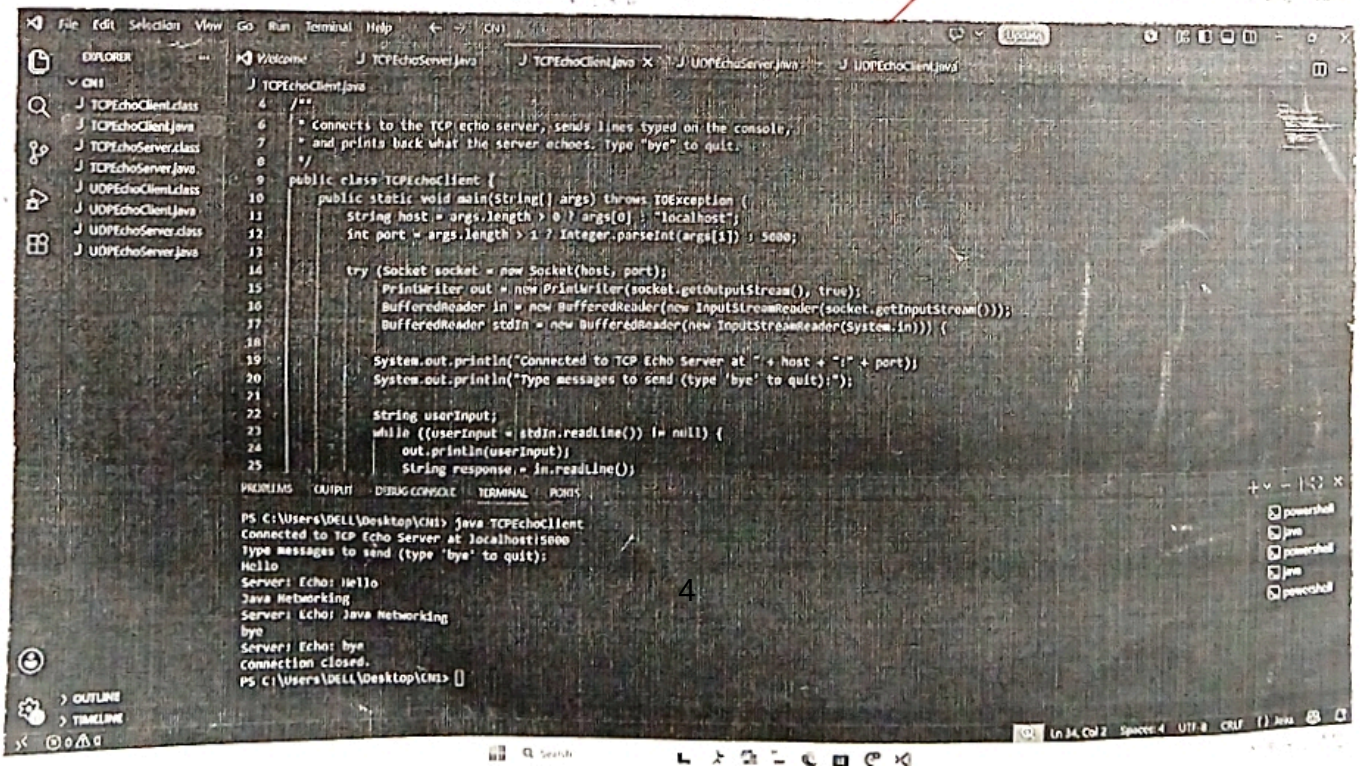
            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                byte[] sendData = userInput.getBytes();
                DatagramPacket request = new DatagramPacket(sendData, sendData.length, address, port);
                socket.send(request);

                byte[] buffer = new byte[1024];
                DatagramPacket response = new DatagramPacket(buffer, buffer.length);
                try {
                    socket.receive(response);
                    String reply = new String(response.getData(), 0, response.getLength());
                    System.out.println("Server: " + reply);
                } catch (SocketTimeoutException e) {
                    System.out.println("No response received (timeout).");
                }
```

```
PS C:\Users\DELL\Desktop\CN1> java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello
Server: Echo: Hello
UDP Communication
Server: Echo: UDP Communication
bye
Server: Echo: bye
Client finished.
PS C:\Users\DELL\Desktop\CN1>
```


TCPEchoServer.java
public class TCPEchoServer {
public static void main(String[] args) throws IOException {
try (ServerSocket serverSocket = new ServerSocket(port)) {
System.out.println("TCP Echo Server listening on port " + port);
while (true) {
try (Socket clientSocket = serverSocket.accept()) {
System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());
BufferedReader in = new BufferedReader(
new InputStreamReader(clientSocket.getInputStream()));
PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
String line;
while ((line = in.readLine()) != null) {
System.out.println("Received: " + line);
out.println("Echo: " + line);
if (line.equalsIgnoreCase("bye")) {
break;
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\CN1> javac TCPEchoServer.java
>> javac TCPEchoClient.java
>> javac UDPEchoServer.java
>> javac UDPEchoClient.java
PS C:\Users\DELL\Desktop\CN1>


TCPEchoServer.java
public class TCPEchoServer {
public static void main(String[] args) throws IOException {
try (ServerSocket serverSocket = new ServerSocket(port)) {
System.out.println("TCP Echo Server listening on port " + port);
while (true) {
try (Socket clientSocket = serverSocket.accept()) {
System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());
BufferedReader in = new BufferedReader(
new InputStreamReader(clientSocket.getInputStream()));
PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
String line;
while ((line = in.readLine()) != null) {
System.out.println("Received: " + line);
out.println("Echo: " + line);
if (line.equalsIgnoreCase("bye")) {
break;
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\CN1> java TCPEchoServer
TCP Echo Server listening on port 5000
Client connected: /127.0.0.1:51758
Received: Hello
Received: Java Networking
Received: bye
Client disconnected: /127.0.0.1:51758


TCPEchoClient.java
/**
* Connects to the TCP echo server, sends lines typed on the console,
* and prints back what the server echoes. Type "bye" to quit.
*/
public class TCPEchoClient {
public static void main(String[] args) throws IOException {
String host = args.length > 0 ? args[0] : "localhost";
int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;
try (Socket socket = new Socket(host, port);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {
System.out.println("Connected to TCP Echo Server at " + host + ":" + port);
System.out.println("Type messages to send (type 'bye' to quit):");
String userInput;
while ((userInput = stdIn.readLine()) != null) {
out.println(userInput);
String response = in.readLine();
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\CN1> java TCPEchoClient
Connected to TCP Echo Server at localhost:5000
Type messages to send (type 'bye' to quit):
Hello
Server: Echo: Hello
Java Networking
Server: Echo: Java Networking
bye
Server: Echo: bye
Connection closed.
PS C:\Users\DELL\Desktop\CN1>